

Build an LLM app *with Python.*

Connect a model to a useful workflow.

Validate the result. Handle the unexpected.

🕒 22-27 min core reading · Offline lab + optional live API · 12 short sections

An LLM application is ordinary software wrapped around a probabilistic model. The model drafts an answer; your code decides what to send, what to accept, how to handle failure, and what the user sees.

WHAT YOU WILL BE ABLE TO DO

Build a small Python command-line app with an interchangeable model provider, a structured response contract, explicit failure states, and local tests. Explain which checks establish correctness of the code and which require reviewing real model outputs.

Reading path: Turn Chapter 9's learner preferences and fictional study-note catalog into a reasoned recommendation. Run the complete workflow offline with prepared responses; optionally connect the same code to a live API. The website itself does not make model calls.

1. Build one useful vertical slice

A **vertical slice** is one small task working from input to visible result. Our task is: take learner preferences and note entries, produce a structured suggestion, and show it for review. We begin with a **command-line interface (CLI)**: a Python program you run from a terminal. A web interface can later call the same application logic.

This is **inference**: using an existing model to produce a response. We are not training weights, building a search index, or creating an autonomous agent. No framework is required for this first workflow. Python's standard library can read JSON, make HTTPS requests, validate results, and run tests.

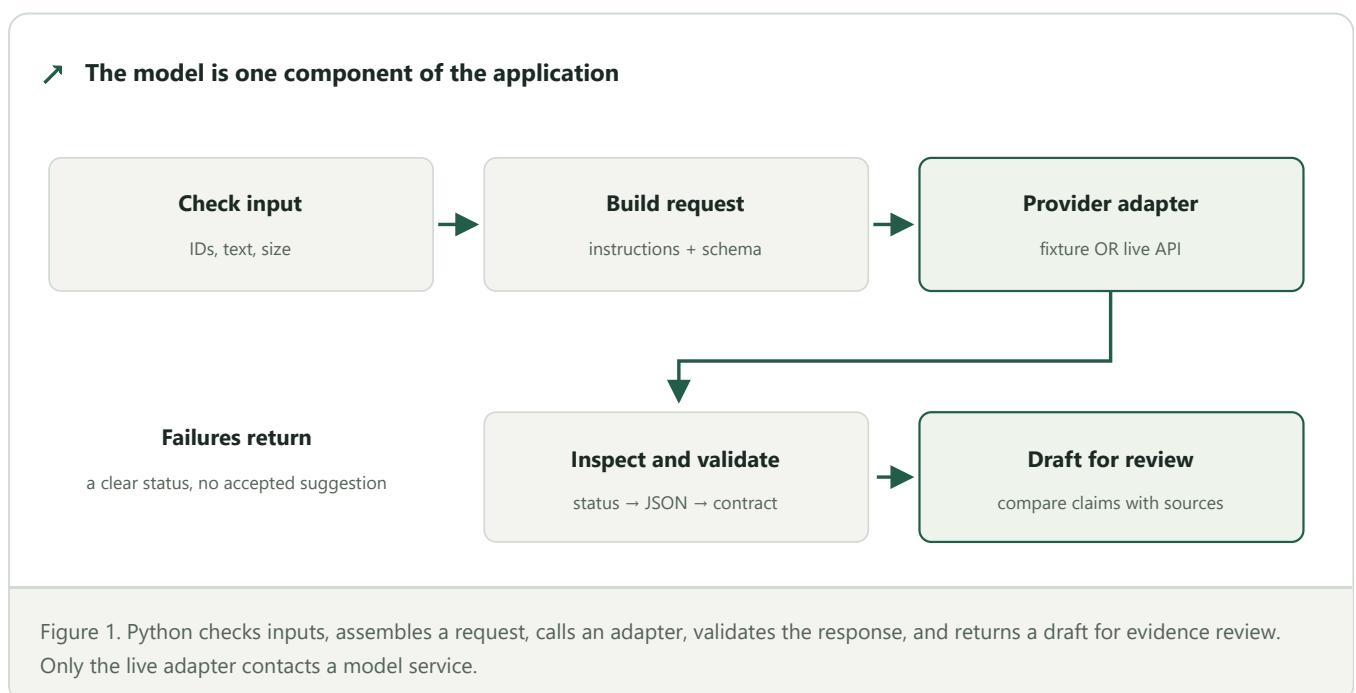
This first learning-assistant app uses supplied preferences and note records to guide an LLM's suggestion and explanation. It does not retrieve from a large collection, train a model, create a quiz, or change a calendar yet.

Reuse Chapter 9's fictional study-session case: S1 asks for an introductory note about model learning, estimated at no more than 30 minutes. S2 lists Overfitting, an introductory learning note estimated at 20 minutes, and Embeddings, an advanced search note estimated at 45 minutes. S3 says Overfitting's exam date is not supplied. A useful suggestion explains why Overfitting fits and keeps the exam date unknown. The learner profile, estimates, and dates are teaching examples.

The result has two layers: the **application status**, such as `review_required`, and an optional **candidate recommendation** with `recommendation`, `exam_date`, `evidence_ids`, and `open_questions`. A failure returns a status without an accepted candidate. This prevents "something went wrong" from masquerading as a valid study suggestion.

2. Separate the workflow from the provider

An **adapter** translates between your application and an external interface. Here the workflow asks a provider to generate a response, then processes what comes back. It does not need to know whether that provider returns a prepared fixture or contacts a model service.



A **fixture** is prepared test data. Our offline provider returns authored responses, including deliberately broken ones. The validation and control flow are real Python execution; the language-model behavior is simulated. This lets us test rare failures repeatedly without network access or charges.

Workbook file	Responsibility
app.py	Validate sources, build requests, run the workflow, expose the CLI
provider.py	Make the optional HTTPS call and classify transport errors
errors.py	Share one provider-error type between modules
fixtures.json / sources.json	Authored outcomes / fictional input records
test_app.py	Check the workflow and adapter using local test doubles
README.md	Run instructions, boundaries, and live setup

Dependency injection means supplying a dependency from outside a function. The important call is `run_recommender(sources, provider, max_attempts=1)`. Tests pass a fixture provider; live mode passes an API provider. Changing that argument changes where responses come from without duplicating the rest of the app.

▼ Go deeper: functions, adapters, SDKs, and frameworks

The provider interface is deliberately small: a `model` name and a `generate(request)` method. A **test double** is a substitute for a dependency used in a test; our fixture provider is one. A real provider adapter must normalize its actual API into the envelope expected by the workflow. Another vendor cannot simply be swapped in by changing an endpoint string.

An **SDK** is a library that wraps a service's protocol in language-friendly objects and methods. It may handle authentication headers, serialization, typed responses, and retries. Our teaching adapter uses Python's built-in HTTP facilities so those boundaries remain visible and offline setup needs no packages. The official OpenAI libraries are an alternative when building further; inspect their retry behavior before adding an outer retry loop.

A web framework handles concerns such as routes and request parsing. An orchestration framework can help coordinate multi-step model workflows. Neither removes the need for a clear contract, evidence checks, or failure handling. Add a dependency when its benefit is concrete.

3. Run the offline app before configuring a service

Extract the ZIP, open a terminal in the extracted folder, and use **Python 3.10 or newer**. These commands use Windows' `py` launcher; substitute your working `python` or `python3` command when needed.

```
py app.py
py app.py --case transient --attempts 2
py -m unittest -v test_app.py
```

The first command returns an authored draft with `status: review_required`. The second runs a prepared temporary-failure case followed by a usable response. Offline waits are recorded but skipped. The third checks the code locally. None of these commands calls a model service, needs credentials, or writes a recommendation file.

Read the output's `mode`, `attempts`, `candidate`, and `trace`. The trace is a list of application events, not the model's internal reasoning. To try different notes, save an array of `id / text` records and pass `--input sources.json`. In offline mode, changing the sources does not make the fixture intelligently adapt; its answer remains authored data.

▼ Go deeper: environments, secrets, and optional live mode

A **virtual environment** isolates Python dependencies for a project. You can create one with `py -m venv .venv`, then run `.venv\Scripts\python.exe app.py` on Windows. This workbook needs no third-party Python packages. If you later add an SDK, install it into the environment and record a tested version.

Live mode requires an API account with access to a model that supports the Responses API and strict JSON-schema output. Set `OPENAI_API_KEY` and `OPENAI_MODEL` in your local process environment, then run `py app.py --live --input sources.json`. The README includes a masked key-entry example for PowerShell 7. A model ID is configuration: confirm its access, capabilities, limits, and price in the provider's current documentation. No particular model is assumed by this workbook.

The explicit `--live` command sends the selected sources to the service and may incur API charges. Begin with the fictional records. Keep credentials in the local runtime or a secret manager, never browser JavaScript, source control, recommendation JSON, or logs. The handbook has no API-key form. Environment variables are a configuration mechanism, not a complete secret-management system.

The adapter requests `store: false`; that option does not mean no network transfer or guarantee zero provider retention. Review the service's data controls before using real workplace records. No live model call was made to validate this chapter; the adapter was tested with substituted HTTP responses.

4. Build a request from validated sources

Validate the input before paying for a model call. Our small app allows one to five records, each with a unique source ID such as S1 and a nonempty text field of at most 2,000 characters. Those are **application limits**, chosen for this exercise, not a model's token limit. A character count is not token accounting.

The request contains stable instructions, source data serialized with `json.dumps`, a JSON schema, and an output-token limit. **Serialization** converts a Python value into a transport representation. It correctly escapes quotes and newlines; it does not make hostile source instructions safe for a model to follow.

The following excerpt is from `build_request`; the complete schema is in the workbook:

```

return {
    "model": model,
    "instructions": INSTRUCTIONS,
    "input": [{"role": "user", "content": json.dumps({"sources": sources})}],
    "text": {"format": {
        "type": "json_schema", "name": "study_suggestion",
        "strict": True, "schema": schema,
    }},
    "max_output_tokens": 1200,
    "store": False,
}

```

The schema requires all four recommendation fields, rejects additional properties, and limits `evidence_ids` to supplied IDs using an **enum**, a list of permitted values. The `exam-date` field permits a string or `null`. `null` means exam timing is not established here; it is different from the string `"null"`.

The local validator also checks nonempty strings and distinct evidence IDs. Even schema-constrained generation is only one layer: it cannot establish that the chosen source proves a claim. The exact request format and supported schema features are documented in [Structured Outputs](#).

5. Make the service boundary explicit

The live adapter sends an HTTPS **POST** to the `Responses` endpoint. The JSON body contains the request; an authorization header carries the API credential. A **timeout** limits waiting on a network operation. The workbook passes 30 seconds to its HTTP transport; this is a network-operation setting, not a guaranteed 30-second deadline for the complete application.

```

request = Request(
    ENDPOINT,
    data=json.dumps(payload).encode("utf-8"),
    headers={"Authorization": "Bearer " + self.api_key,
             "Content-Type": "application/json"},
    method="POST",
)
with self.transport(request, timeout=30) as response:
    return load_json(response.read().decode("utf-8"))

```

Read this as four transformations: Python dictionary → JSON text → UTF-8 bytes → HTTP request. The return path reverses that process into a response dictionary. The snippet shows the successful path; the actual adapter also handles HTTP errors, connection failures, and invalid response JSON. The [Responses API reference](#) defines the envelope and request options.

Notice the two JSON layers. The **API envelope** contains status and output items. One assistant output item contains **candidate recommendation text**, which must itself be parsed as JSON. Parsing the outer envelope does not validate the inner recommendation. Never assume the first output item is the answer: a reasoning item can appear before an assistant message.

6. Accept a draft only after distinct checks

The workflow applies checks in order, stopping at the first unusable result:

Check	Failure example	App outcome
Input	No source records	invalid_input, zero provider calls
Transport/service	Credentials rejected or retry budget exhausted	service_error
Envelope	Incomplete response or refusal	incomplete or refused
Candidate JSON	Broken syntax, duplicate keys, NaN	invalid_json
Recommendation contract	Wrong types or unknown source IDs	invalid_recommendation
Evidence review	Supported structure but invented exam timing	Still review_required; human checks meaning

An HTTP success status does not guarantee a complete model answer. If generation stops early, do not add a brace to the truncated text and call it finished. If the model refuses, handle that as a distinct outcome rather than parsing the refusal as recommendation data or trying to bypass it.

The application deliberately calls its accepted state **review_required**, never “verified.” In the unsupported-claim fixture, `exam_date` says 2026-12-01 and cites S1. S1 exists, but only states learner preferences; it says nothing about exam timing. The type and ID checks pass; the evidence claim fails.

▼ Go deeper: structured outputs, rendering, and actions

Our malformed and wrong-type fixtures deliberately inject bad outcomes. They test defensive application behavior; they are not measurements of how often schema-constrained models fail. A completed, non-refused response under a supported strict schema has stronger format guarantees, but can still contain incorrect values. Provider refusal and incomplete-response behavior must be handled separately.

Treat the candidate as data. Render text with an escaping UI mechanism; do not feed it into raw HTML, SQL strings, `eval`, or shell commands. This app grants no model-generated action permissions. It only prints a candidate and its status. If a future app saves a recommendation or creates a calendar event, the application must authorize that separate action and handle duplicate submissions.

7. Retry temporary failures with a budget

A **retry** repeats a request after failure. It can help when a service is briefly unavailable; it cannot repair an invalid credential or establish a missing fact. Two total attempts means the original call plus **one retry**.

➤ A retry needs a reason and a stopping point

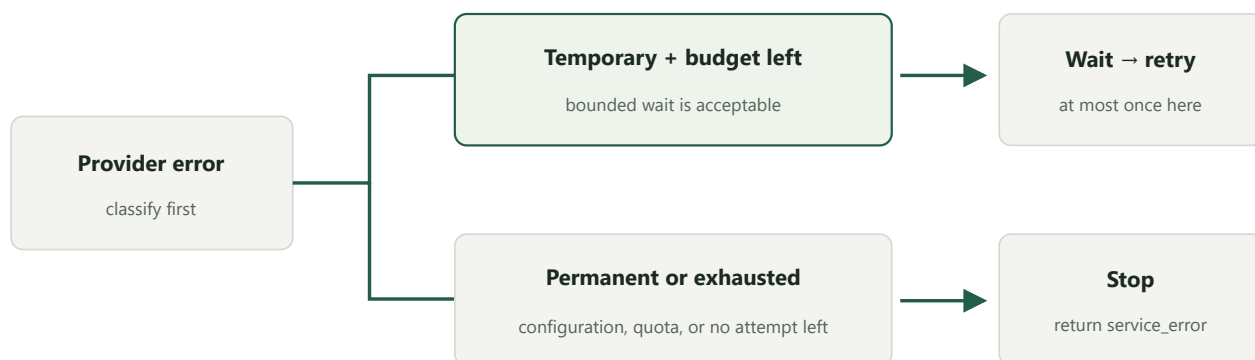


Figure 2. The teaching app allows one or two total attempts. It retries only selected temporary provider failures. Invalid content, refusals, and permanent configuration failures stop.

The workbook defaults to one attempt. With `--attempts 2`, it permits one retry for selected temporary server errors, connection/timeouts, or explicitly recognized rate-limit errors. Authentication errors, invalid requests, quota errors, unknown 429 codes, and content-validation failures stop. A 429 may mean throttling or a spending limit; inspect the provider's error code rather than treating every 429 alike. See the [official error-code guide](#).

Before its one retry, the live adapter uses a one-second default wait or a numeric `Retry-After` delay of at most ten seconds. Longer or unrecognized delay instructions cause a stop so the caller can retry later. Offline mode records the same decision without sleeping. This tiny policy makes the attempt budget observable; it is not a complete high-volume retry system.

▼ Go deeper: backoff, deadlines, and uncertain outcomes

Exponential backoff increases successive waits, for example 1, 2, 4 seconds; **jitter** adds variation so many clients do not retry together. This workbook permits only one retry, so it has no growing sequence or jitter. A production system also needs a total deadline, concurrency limits, cancellation, and a deliberate policy for provider guidance.

A timeout means the client did not receive a usable answer in time. The service may still have processed the request. A retry can generate another response and another charge. If a workflow later performs an external action, a retry must not blindly repeat that side effect. **Idempotency** means repeating an operation has the same intended effect as doing it once; it needs explicit application or service support, not a prompt instruction.

If an SDK already retries requests, an outer loop can multiply the number of calls. Choose one place to own the retry policy and measure the actual attempts. An incomplete generation may need a different output budget, while malformed content may need a prompt or schema change; neither is automatically solved by a transport retry.

8. Trace the application through success and failure

Choose a case and total-attempt budget. Move through the recorded events and compare the final status. These traces were generated by running the downloadable Python workflow against authored fixtures. The browser replays the recordings; it does not execute Python or contact an LLM.

FOLLOW THE CODE'S DECISIONS

Recorded Python · fixture responses

One workflow. Different outcomes.

Replay real Python control flow with authored service responses. No model runs here; offline retry waits were skipped.

Recorded outcomes from the Python workbook; two total attempts are allowed below. The first case also passes with one attempt. Temporary failure stops after one attempt, but reaches review_required with two.

Prepared case	Final status	Calls
Usable draft	review_required	1
Temporary service failure	review_required	2
Invalid credentials	service_error	1
Refused response	refused	1
Incomplete response	incomplete	1
Broken JSON	invalid_json	1
Wrong field type	invalid_recommendation	1
Unknown source ID	invalid_recommendation	1
Unsupported claim	review_required	1
Empty source list	invalid_input	0

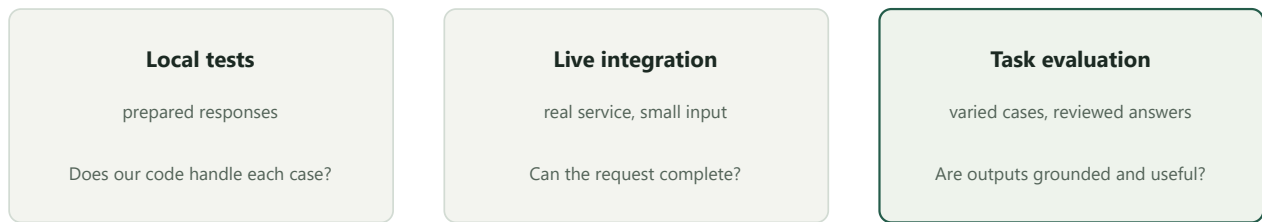
Key comparison: the unsupported-claim case still reaches review_required. A false claim can pass types and source-ID checks. The empty source list stops before any provider call.

Try **Temporary service failure** with one attempt, then two. Next try **Unsupported claim**. The first comparison shows a real control-flow difference; the second shows why reaching the accepted structural state does not establish evidence quality. Changing the budget does not alter a prepared model answer.

9. Test behavior, then evaluate useful answers

Unit tests check a small piece such as parsing or validation. A **workflow test** checks several pieces together with controlled dependencies. A **live integration test** checks whether the real provider connection works. An **evaluation** assesses the quality of outputs on task-specific cases. Keep these claims separate.

➤ Different tests answer different questions



A working connection does not establish answer quality.

This chapter ships local checks; live quality depends on the chosen model and task.

Figure 3. Local tests establish application behavior. A live integration test checks the provider connection. Task evaluations assess the usefulness and grounding of real model outputs. Passing one layer does not establish the others.

The workbook tests zero calls for invalid input, a maximum of two calls for temporary failures, refusal handling, malformed output, field rules, and the known limitation of unsupported claims. Adapter tests intercept HTTP locally to inspect headers, request data, timeout settings, and error classification. They use a dummy credential and never call the service.

After choosing a live model, evaluate real outputs on varied study requests: supplied and missing exam timing, conflicting study-time estimates, changing preferences, and embedded instructions. Review factual support and completeness against sources, not exact wording. Keep some cases separate from prompt revision and repeat trials when response variability matters. Passing fixture tests is a prerequisite for that work, not evidence that the model is accurate.

▼ Go deeper: observability, cost, streaming, and state

Observability helps explain what the app did. The workbook records a prompt version, selected model label, attempts, outcome, and stage trace. These events exclude raw source bodies and credentials; the printed candidate is still content you must handle appropriately. A deployed service can additionally record request IDs, elapsed time, token usage, and failure rates with suitable access and retention controls. This workbook does not invent latency, token, or cost measurements for fixture runs.

For a simple uncached text request, a planning estimate is $\text{input tokens} \times \text{input rate} + \text{output tokens} \times \text{output rate}$, using consistent pricing units. Actual billing can include cached-input rules, reasoning tokens, tools, and retries. Check the chosen model's current pricing and inspect returned usage; words and characters are not exact token counts.

Streaming delivers pieces as they become available. It improves perceived responsiveness, but an unfinished stream is not a validated recommendation. If you stream a draft to a UI, label it pending and wait for the final status before accepting it. The workbook uses a non-streaming request so the response boundary is easier to inspect.

Conversation state is the earlier information included or referenced in later calls. This app is intentionally stateless across runs: it sends the chosen sources each time and does not pass a previous response ID. A printed trace is not conversational memory. A future chat interface must choose what history to retain, how to bound it, and how to isolate one user's state from another's.

10. Practice: explain what the app guarantees

Exercise 1: follow the retry budget

Run the temporary-failure fixture with one attempt and then two. Predict the final status, provider-call count, and retry count for each.

▼ Reveal the trace

One attempt returns `service_error` after one call and zero retries. Two attempts return `review_required` after two calls and one retry. The fixture's first outcome is a temporary 503 and its second is a usable authored draft. Offline waiting is skipped; the control-flow decisions still run in Python.

Exercise 2: locate the missing check

The unsupported-claim fixture passes JSON, type, and ID checks. Which statement is false, which source fails to support it, and what should the reviewed recommendation contain instead?

▼ Reveal the evidence check

The claim that Overfitting has an exam on 2026-12-01 is unsupported. S1 only states learner preferences. `exam_date` should remain null and exam timing should be an open question. No additional string-type check can prove a missing catalog fact.

Exercise 3: distinguish two failures

An API call returns HTTP 200 with an incomplete envelope. A second scenario returns HTTP 401. Why should neither produce an accepted recommendation, and what kind of investigation follows each?

▼ Reveal the application decision

The incomplete response does not contain a finished answer: inspect the completion reason and request limits before rerunning. The 401 indicates an authentication failure: inspect credentials and access rather than retrying unchanged input. The first fails at the envelope layer; the second fails at the service layer. Both return no accepted candidate.

11. Check your understanding

Answer all five questions, then read the explanations.

01 What changes when the app switches from the fixture provider to the live provider?

- A. The model is trained on the fixture answers.
- B. The request crosses a real service boundary; the surrounding validation workflow stays the same.
- C. Source review is no longer necessary.

Answer: B. An adapter changes how a response is obtained. Prepared fixture outputs do not train a model, and live responses still need validation and evidence review.

02 The server returns HTTP 200 with an incomplete model response. What should happen?

- A. Stop with an incomplete status and do not accept the partial recommendation.
- B. Treat 200 as proof that the recommendation is complete.
- C. Append a closing brace and approve it.

Answer: A. HTTP success describes the request at one layer. The response envelope, generated JSON, recommendation contract, and evidence are separate checks.

03 With two total attempts, the first request gets a temporary 503 and the second succeeds. How many retries happened?

- A. Two.
- B. Zero.
- C. One.

Answer: C. Total attempts include the original call. The policy permits one original attempt plus one retry, not two retries.

04 A recommendation contains an invented exam date and valid source IDs. Its status is `review_required`. What does that mean?

- A. The exam date has been verified.
- B. The code accepted the structure; a person must still compare claims with the sources.
- C. The model is certain about every claim.

Answer: B. A type check and an ID membership check cannot establish whether a source supports a claim. The app deliberately never labels this status verified.

05 What do the offline fixture tests establish?

- A. The chosen live model is accurate and available.
- B. The real API will always return the fixture answer.
- C. The application handles the prepared inputs and response paths as intended.

Answer: C. Fixture tests exercise application behavior. Live integration and task evaluation are additional kinds of evidence, and neither was replaced by these local tests.

12. Keep the model call small and the application explicit

You now have an executable path from source records to a reviewable draft, with the service call isolated behind an adapter. The strongest practical habit is to name the guarantee at each boundary: valid input, completed response, valid JSON, accepted structure, and reviewed evidence are different achievements.

▼ Quick-reference glossary

Term	Recall cue
CLI / vertical slice	Terminal interface / one task working end to end
Adapter / dependency injection	Translate an interface / supply a dependency externally
Fixture / test double	Prepared test data / substitute dependency used for testing
Serialization / envelope	Encode values for transport / service response wrapper
SDK / virtual environment	Service client library / isolated Python dependencies
Timeout / retry budget	Bound a wait / bound the number of attempts
Backoff / jitter	Increase retry waits / vary them across clients
Idempotency	Repetition preserves the intended effect
Trace / observability	Recorded events / ability to inspect application behavior
Integration test / eval	Check connected components / assess task output quality
Streaming / state	Receive incremental output / retain context across interactions

Sources and fixtures are original, fictional teaching material. Local tests verify code paths; they do not establish live API availability, pricing, or model quality. Provider documentation linked beside relevant claims was checked for this edition. The Python source and recorded traces ship with the same chapter.

Next: Chapter 11, Project 1: a personal learning assistant, combines the workflow with a usable interface and a task evaluation.